

EMAIL SPAM CLASSIFIER

PRESENTED BY:

ASHISH RANJAN - 2019UEC2677

ASHISH KUMAR - 2019UEC2669

SAGAR GUPTA - 2019UEC2721

MANISH DRALL - 2019UEC2671

Email Spam Classification Application

Classification

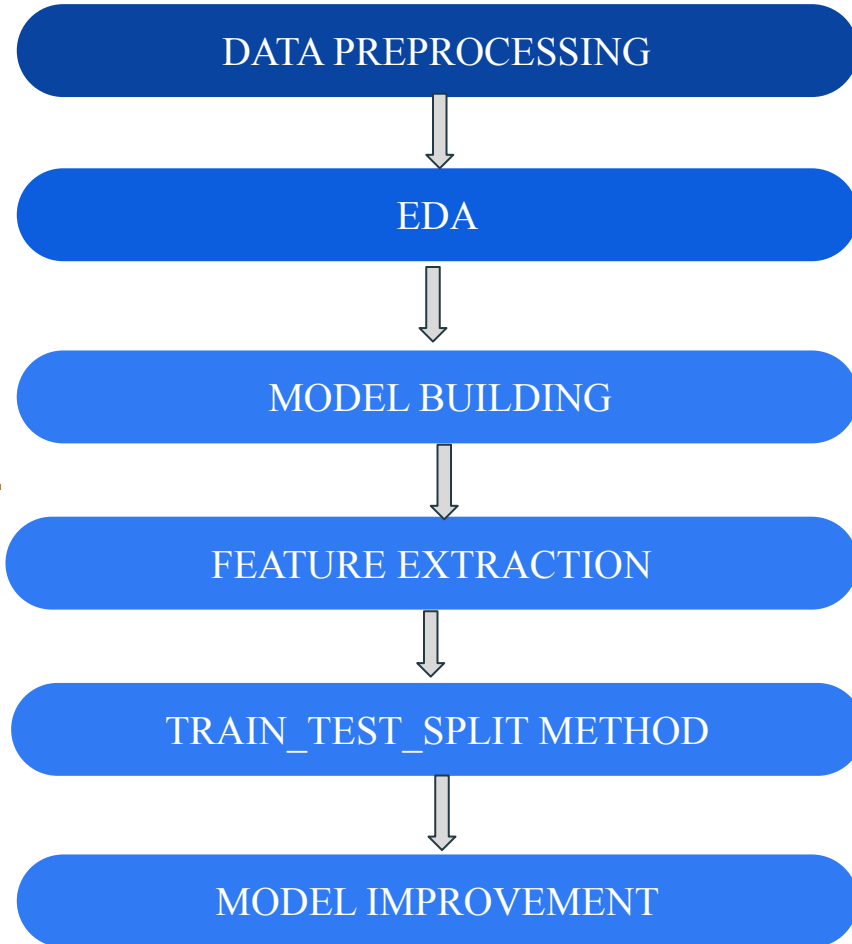
Enter a text

you got free entry in fifa final"

Process

This is A Spam Email

FLOW OF PRESENTATION



DATA PREPROCESSING

- **Data collection:** Collect a dataset of email messages, labeled as spam or non-spam. You can use publicly available datasets or create your own.
- **Data cleaning:** Remove any irrelevant information from the emails, such as email addresses, names, dates, and other metadata. Also, remove any HTML tags or formatting codes that may be present in the emails.
- **Tokenization:** Split the emails into individual words or tokens. You can use Python's NLTK library or other tokenization tools to perform this task.
- **Stopword removal:** Remove commonly used words that do not add any significant meaning to the emails, such as "the," "a," "and," etc.
- **Feature extraction:** Convert the preprocessed text data into numerical features that can be used by machine learning algorithms. Popular feature extraction techniques for text data include bag-of-words, TF-IDF, and word embeddings.
- **Splitting the dataset:** Split the preprocessed dataset into training and testing sets, ensuring that the ratio of spam and non-spam emails is the same in both sets.

●

EDA

- Data Exploration: Explore the dataset to get a better understanding of the distribution of spam and non-spam emails. Calculate the percentage of spam emails in the dataset.
- Data Visualization: Create visualizations such as histograms, bar charts, or pie charts to represent the distribution of spam and non-spam emails in the dataset. This will help identify any imbalances or patterns in the data.
- Word Frequency Analysis: Analyze the most frequent words in spam and non-spam emails. This can help identify words or phrases that are specific to spam emails and can be used as features in the model.
- Length Analysis: Analyze the length of spam and non-spam emails. This can help identify any differences in length between the two types of emails, which can be used as features in the model.
- Correlation Analysis: Analyze the correlation between different features in the dataset. This can help identify any relationships between features that may be useful in the model.

MODEL BUILDING

FEATURE EXTRACTION

1) COUNT VECTORIZER (BAG OF WORDS)

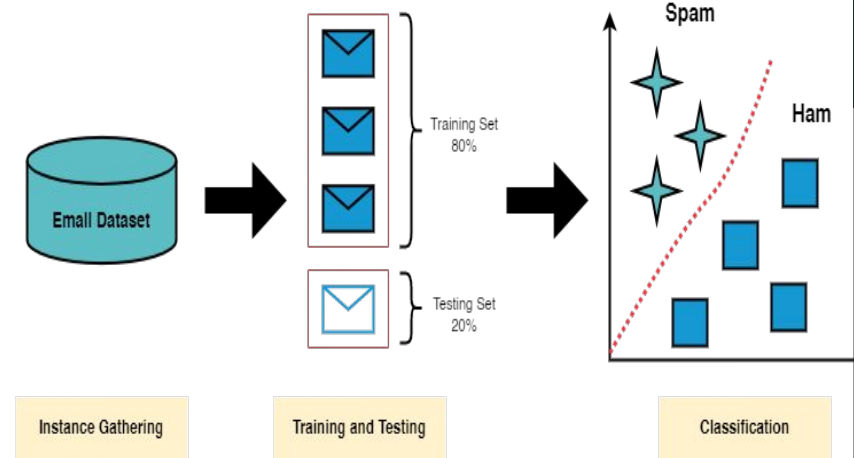
- CountVectorizer is a technique used for converting text data into numerical representations that can be used in machine learning models.
- It works by creating a vocabulary of all the unique words in the text data and then counting the occurrence of each word in each document.
- Each document is then represented as a vector of word counts, with each dimension of the vector representing a unique word in the vocabulary.
- This results in a high-dimensional, sparse matrix where each row corresponds to a document and each column corresponds to a word in the vocabulary.
- CountVectorizer can be customized by specifying options such as the minimum document frequency or maximum number of features.

2) TF-IDF

- TF-IDF stands for Term Frequency-Inverse Document Frequency. It is a commonly used technique in natural language processing and information retrieval to represent text data as numerical vectors.
- In TF-IDF vectorization, each document is represented as a vector of numerical values, where each dimension of the vector corresponds to a unique term in the entire corpus. The value of each dimension in the vector represents the importance of that term in the document.
- The TF-IDF value of a term in a document is calculated as the product of two values: the term frequency (TF) and the inverse document frequency (IDF). The term frequency is simply the number of times a term appears in a document, while the inverse document frequency is a measure of how important the term is in the entire corpus, calculated as the logarithm of the ratio of the total number of documents to the number of documents that contain the term.

TRAIN_TEST_SPLIT METHOD

- Split the data: The first step is to split the dataset into two parts - a training set and a testing set. The training set is used to train the model, while the testing set is used to evaluate the performance of the model.
- Training the model: The training set is used to train the model. The model is fitted to the training data by adjusting the model parameters to minimize the error between the predicted and actual values.
- Testing the model: Once the model has been trained, it is used to make predictions on the testing set. The predicted values are compared to the actual values in the testing set to evaluate the performance of the model.



NAIVE BAYES ALGORITHM

- Naive Bayes is a probabilistic machine learning algorithm that is widely used for classification tasks, including spam detection, sentiment analysis, and document categorization.
- The algorithm is based on Bayes' theorem, which describes the probability of an event given its prior probability and the probability of the evidence. In the context of classification, the Naive Bayes algorithm estimates the probability of each class (e.g. spam or non-spam) given a set of features (e.g. words in an email).
- The "naive" in Naive Bayes refers to the assumption that the features are conditionally independent, which means that the presence of one feature does not affect the probability of another feature. This assumption simplifies the calculations and makes the algorithm computationally efficient.
- There are several types of Naive Bayes classifiers, including Gaussian Naive Bayes, Multinomial Naive Bayes, and Bernoulli Naive Bayes, which differ in their assumptions about the distribution of the features.

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

using Bayesian probability terminology, the above equation can be written as

$$\text{Posterior} = \frac{\text{prior} \times \text{likelihood}}{\text{evidence}}$$

NAIVE BAYES CLASSIFIER

```
graph TD; A[NAIVE BAYES CLASSIFIER] --> B[GAUSSIAN]; A --> C[BEROULLI]; A --> D[MULTINOMIAL]
```

GAUSSIAN

BEROULLI

MULTINOMIAL

```
In [339]: gnb = GaussianNB()
          mnb = MultinomialNB()
          bnb = BernoulliNB()
```

```
In [340]: gnb.fit(X_train,y_train)
          y_pred1 = gnb.predict(X_test)
          print(accuracy_score(y_test,y_pred1))
          print(confusion_matrix(y_test,y_pred1))
          print(precision_score(y_test,y_pred1))
```

```
0.874274661508704
[[791 105]
 [ 25 113]]
0.518348623853211
```

```
In [341]: mnb.fit(X_train,y_train)
          y_pred2 = mnb.predict(X_test)
          print(accuracy_score(y_test,y_pred2))
          print(confusion_matrix(y_test,y_pred2))
          print(precision_score(y_test,y_pred2))
```

```
0.9593810444874274
[[896   0]
 [ 42  96]]
1.0
```

VERY GOOD
PRECISION

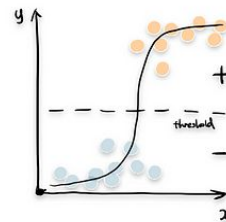
```
In [342]: bnb.fit(X_train,y_train)
          y_pred3 = bnb.predict(X_test)
          print(accuracy_score(y_test,y_pred3))
          print(confusion_matrix(y_test,y_pred3))
          print(precision_score(y_test,y_pred3))
```

```
0.971953578336557
[[894   2]
 [ 27 111]]
0.9823008849557522
```

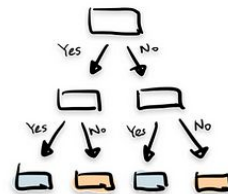
SOME MORE ML ALGORITHMS

- LOGISTIC REGRESSION
- SVC
- DECISION TREE CLASSIFIER
- K NEIGHBOUR CLASSIFIER
- RANDOM FOREST CLASSIFIER
- ADABOOST CLASSIFIER
- BAGGING CLASSIFIER
- EXTRA TREES CLASSIFIER
- GRADIENT BOOSTING CLASSIFIER
- XGB CLASSIFIER

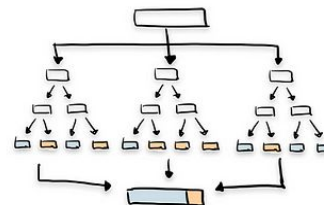
Logistic Regression



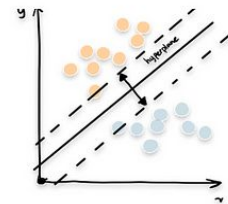
Decision Tree



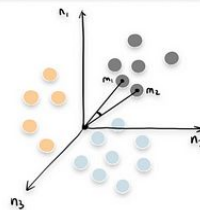
Random Forest



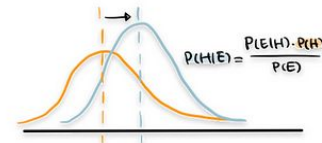
Support Vector Machine



K Nearest Neighbour



Naive Bayes



	Algorithm	Accuracy	Precision
8	ETC	0.977756	0.991453
0	SVC	0.972921	0.974138
10	xgb	0.971954	0.950413
5	RF	0.970019	0.990826
6	AdaBoost	0.962282	0.954128
2	NB	0.959381	1.000000
7	BgC	0.957447	0.861538
4	LR	0.951644	0.940000
9	GBDT	0.951644	0.931373
3	DT	0.935203	0.838095
1	KN	0.900387	1.000000

WHERE :

- SVC : Support Vector Classifier
- DT : Decision Tree Classifier
- KN : K Neighbour Classifier
- RF : Random Forest Classifier
- AdaBoost : AdaBoost Classifier
- BgC : Bagging Classifier
- ETC : Extra Trees Classifier
- GBDT : Gradient Boosting Classifier
- Xgb : XGB Classifier

MODEL IMPROVEMENT

- Max_Feature parameter of TF-IDF

	Algorithm	Accuracy	Precision	Accuracy_max_ft_3000	Precision_max_ft_3000
0	KN	0.900387	1.000000	0.905222	1.000000
1	NB	0.959381	1.000000	0.971954	1.000000
2	ETC	0.977756	0.991453	0.979691	0.975610
3	RF	0.970019	0.990826	0.975822	0.982906
4	SVC	0.972921	0.974138	0.974855	0.974576
5	AdaBoost	0.962282	0.954128	0.961315	0.945455
6	xgb	0.971954	0.950413	0.968085	0.933884
7	LR	0.951644	0.940000	0.956480	0.969697
8	GBDT	0.951644	0.931373	0.946809	0.927835
9	BgC	0.957447	0.861538	0.959381	0.869231
10	DT	0.935203	0.838095	0.931335	0.831683

	Algorithm	Accuracy	Precision	Accuracy_max_ft_3000	Precision_max_ft_3000
0	KN	0.900387	1.000000	0.905222	1.000000
1	NB	0.959381	1.000000	0.971954	1.000000
2	ETC	0.977756	0.991453	0.979691	0.975610
3	RF	0.970019	0.990826	0.975822	0.982906
4	SVC	0.972921	0.974138	0.974855	0.974576
5	AdaBoost	0.962282	0.954128	0.961315	0.945455
6	xgb	0.971954	0.950413	0.968085	0.933884
7	LR	0.951644	0.940000	0.956480	0.969697
8	GBDT	0.951644	0.931373	0.946809	0.927835
9	BgC	0.957447	0.861538	0.959381	0.869231
10	DT	0.935203	0.838095	0.931335	0.831683

- SCALING : By importing MINMAX scalar rather than standard scaler

Accuracy_scaling	Precision_scaling
0.905222	0.976190
0.978723	0.946154
0.979691	0.975610
0.975822	0.982906
0.971954	0.943089
0.961315	0.945455
0.968085	0.933884
0.967118	0.964286
0.946809	0.927835
0.959381	0.869231
0.932302	0.840000

- NUM_CHAR Column

Accuracy_scaling	Precision_scaling	Accuracy_num_chars	Precision_num_chars
0.905222	0.976190	0.928433	0.771186
0.978723	0.946154	0.940039	1.000000
0.979691	0.975610	0.976789	0.975000
0.975822	0.982906	0.974855	0.982759
0.971954	0.943089	0.866538	0.000000
0.961315	0.945455	0.971954	0.950413
0.968085	0.933884	0.970019	0.942149
0.967118	0.964286	0.961315	0.971154
0.946809	0.927835	0.948743	0.929293
0.959381	0.869231	0.968085	0.913386
0.932302	0.840000	0.943907	0.877358

THANK YOU.....